

Python

Amitabha Sanyal

June 2026

Basic Python

Python: Where It Fits

- **Scripting and automation**

- Python can directly manipulate files and directories (`shutil`), search pathnames (`rglob`), parse text/CSV/JSON (`csv.DictReader`), and access OS services (`os.getcwd`, `os.environ`, `os.system`, `os.makedirs`) through its standard library.
- It can also orchestrate external programs using `subprocess`.

- **Data analysis and scientific computing**

- `numpy`, `scipy`, `pandas`, `matplotlib`.

- **Web services and APIs**

- `Flask`, `Django`, `FastAPI`, `requests`.

- **Machine learning and AI**

- `scikit-learn`, `PyTorch`, `TensorFlow`.

- **Glue/Wrapper language**

- Python often provides the control layer over C/C++/Fortran/Rust libraries and tools.
Examples: `numpy`, `scipy`, the `gdb-Python` API.

References: [Official documentation](#), [Python Tutorial: Brief Tour of the Standard Library](#), and [PyPI: The Python Package Index](#)

Getting Python to Run

- On Linux, use the OS package manager for the system Python:

```
1 $ sudo apt update
2 $ sudo apt install python3 python3-pip python3-venv
```

For the newest Python, do not overwrite system Python; install separately or build from source.

- Check the interpreter:

```
1 $ python3 --version
2 $ python3 -m pip --version
```

- Run Python in three common ways:

```
1 $ python3           # interactive prompt / REPL
2 $ python3 program.py # run a script file
3 $ python3 -m module_name # run a module as a program
```

Script or Module? The `__main__` Convention

```
1 # collatz.py
2 def collatz(n):
3     while n > 1:
4         print(n, end=" ")
5         n = 3*n + 1 if n % 2 else n // 2
6     print(1)
7 def main():
8     n = int(input("Enter n: "))
9     collatz(n)
10 if __name__ == "__main__":
11     main()
```

```
1 # use_collatz.py
2
3 import collatz
4 collatz.collatz(7)
```

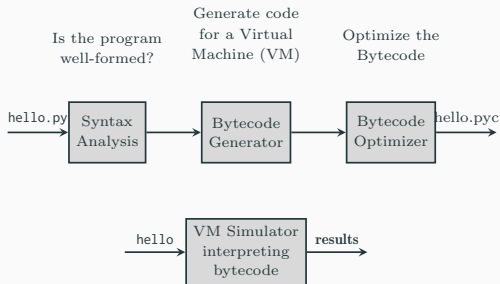
```
1 # Run the script directly, __name__ = "__main__"
2 $ python3 collatz.py
3 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

```
1 # Import collatz.py as a module, __name__ =
  "collatz"
2 $ python3 use_collatz.py
3 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

```
1 # Run in a REPL importing the module, __name__ =
  "collatz"
2 $ python3
3 >>> import collatz
4 >>> collatz.main()
5 ...
```

References: [__main__: top-level code environment](#), [Modules](#).

Execution Model: Python is Interpreted



- Python source is first compiled to bytecode.
- The bytecode is then executed by the Python virtual machine.
- Imported modules may have cached bytecode in `__pycache__`.
- **Bytecode is an implementation detail of CPython.**

References: [Execution model](#), [dis: bytecode disassembler](#).

Bytecode

Source:

```
1  1  import dis
2  ...
3  4  def collatz(n):
4  5      while n > 1:
5  6          print(n, end=' ')
6  7          if (n % 2):
7  8              # n is odd
8  9              n = 3*n + 1
9  10         else:
10 11             # n is even
11 12             n = n//2
12 13     print(1)
13 14     def main():
14 15         dis.dis(collatz)
```

Bytecode: The shaded lines correspond to $n = 3*n + 1$.

```
1  7      20 LOAD_FAST          0 (n)
2      22 LOAD_CONST         4 (2)
3      24 BINARY_MODULO
4      26 POP_JUMP_IF_FALSE    20 (to 40)
5      28 LOAD_CONST            5 (3)
6      30 LOAD_FAST            0 (n)
7      32 BINARY_MULTIPLY
8      34 LOAD_CONST            1 (1)
9      36 BINARY_ADD
10     38 JUMP_FORWARD          3 (to 46)
11  >>    40 LOAD_FAST            0 (n)
12     42 LOAD_CONST            4 (2)
13     44 BINARY_FLOOR_DIVIDE
14  >>    46 STORE_FAST           0 (n)
```

Reference: [dis module documentation](#).

Python Values and Variables

- **C/C++ variable:** denotes a typed storage location.
- **Python variable:** may denote runtime values of different types during execution.
- Therefore, its value cannot, in general, be stored directly in a fixed-size, fixed-type stack location.
- Instead, a Python variable typically stores in stack a reference to a heap-allocated object representing the value.
- The referenced object has its own runtime **identity**, **type**, and **value**.

```
1 >>> a = [10, 20]
2 >>> b = a
3 >>> a is b      # Reference equality
4 True
5 >>> a == b     # Value equality
6 True
```

```
1 >>> c = [10, 20]
2 >>> a is c      # Reference equality
3 False
4 False
5 >>> a == c     # Value equality
6 True
```

References: [Data model: Objects, values and types](#), [Execution model: Naming and binding](#).

Dynamic Type Checking: Consequences

```
1 def fact(n):  
2     if (n < 0):  
3         return "The argument cannot be negative"  
4     elif (n == 0):  
5         return 1  
6     else:  
7         return (n*fact(n-1))  
8  
9 arg=int(input("arg= "))  
10 print(4+fact(arg))
```

- Python does not assign a fixed type to the function fact.
- fact(-1) returns a string.
- 4 + fact(-1) fails only when evaluated.
- Type checking is part of the BINARY_ADD and BINARY_MULTIPLY instructions.
- This is powerful, but errors can hide on untested paths.

References: [Data model: objects, values and types](#), [Errors and exceptions](#), [The CPython virtual machine](#), [Python operators](#).

Python lists: General-purpose sequences

- Lists are ordered.

```
1 >>> [1,2,3,4] == [4,1,3,2]
2 False
```

- Lists are heterogeneous (the same list can have different types of objects).

```
1 >>> a = [21.42, 'foobar', 3, 4, 'bark', False, 3.14159]
2 >>> mixed = [int, collatz, sin, fact]
3 >>> empty = []
```

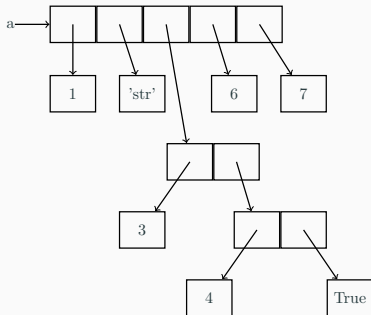
- List elements can be accessed by indices: (0 to 6 or -7 to -1 in the example below)

```
1 >>> a = [21.42, 'foobar', 3, 4, 'bark', False, 3.14159]
2 >>> a[-5]
3 3
4 >>> a[2:4]           # Slice: from a[2] up to, but not including, a[4]
5 [3, 4]
```

Python Lists—Nesting and Representation

- Lists can be nested to arbitrary depth.

```
1 >>> a = [1, 'str', [3, [4, True]], 6, 7]
```



- Lists are represented as dynamic arrays with the varying allocation sizes.
- [This](#) tells you how to find the size of any data object (including lists) in Python.
 - Find the size of `a` in the example above, and explain the observed size.

[Here](#) is another site that gives you the internal implementation of lists in Python.

Python lists—Slicing

- List slicing in Python:
 - `a[:n]` is the same as `a[0:n]`
 - `a[n:]` is the same as `a[n:len(a)]`
 - Thus `a[:n] + a[n:] = a = a[:]` (+ is append)
 - You can also add a stride (step) in a slice.

```
1 >>> a = ['foo', 'bar', 'baz', 'bark', 'qux', 'cor']
2 >>> a[0:5:2]
3 ['foo', 'baz', 'qux']
4 >>> a[5:0]
5 []
6 >>> a[5:0:-1]
7 ['cor', 'qux', 'bark', 'baz', 'bar']
```

List updates and mutability

Each example starts with:

```
1 >>> a = [1, 'str', [3, [4, True]], 6, 7]
2 >>> b = a          # b names the same list object
```

append: add one object

```
1 >>> a.append([8, 9])
2 >>> a
3 [1, 'str', [3, [4, True]], 6, 7,
4  [8, 9]]
5 >>> a is b # True
```

+=: in-place concatenation

```
1 >>> a += [8, 9]
2 >>> a
3 [1, 'str', [3, [4, True]], 6, 7,
4  8, 9]
5 >>> a is b # True
```

insert: insert object at index

```
1 >>> a.insert(2, 'name')
2 >>> a
3 [1, 'str', 'name', [3, [4, True]],
4  6, 7]
5 >>> a is b # True
```

extend: add elements

```
1 >>> a.extend([8, 9])
2 >>> a
3 [1, 'str', [3, [4, True]], 6, 7,
4  8, 9]
5 >>> a is b # True
```

+=: fresh list and reassignment

```
1 >>> a = a + [8, 9]
2 >>> a
3 [1, 'str', [3, [4, True]], 6, 7,
4  8, 9]
5 >>> a is b # False
```

List Comprehension

Python has a powerful feature called list comprehension.

List Comprehension Examples:

```
1 >>> [x*x for x in [1,2,3,4,5,6,7] if x % 2 == 0]
2 [4, 16, 36]
```



```
1 >>> [x+y for x in [1,2,3] for y in [5,6,7]]
2 [6, 7, 8, 7, 8, 9, 8, 9, 10]
```

- A list comprehension can have any number of generators and guards.
- The variables in a guard should either be from the generators preceding it or from the variables in scope outside the list comprehension.

For Loop Equivalents:

```
1 result = []
2 for x in [1,2,3,4,5,6,7]:
3     if x % 2 == 0:
4         result.append(x*x)
5 print(result) # Output: [4, 16, 36]
```

```
1 result = []
2 for x in [1,2,3]:
3     for y in [5,6,7]:
4         result.append(x+y)
5 print(result) # Output:
[6,7,8,7,8,9,8,9,10]
```

List Comprehension

Here is quicksort using list comprehension

```
1 def qsort(lst):  
2     if lst == []:  
3         return []  
4     pivot = lst[0]  
5     l = qsort([x for x in lst[1:] if x < pivot])  
6     u = qsort([x for x in lst[1:] if x >= pivot])  
7     return l + [pivot] + u
```

Manual Memory Management in C and C++

```
1 void process(void) {  
2     int *p = malloc(100 * sizeof(int));  
3     if (p == NULL)  
4         return;  
5     if (!read_data(p))  
6         return;           // memory leak  
7     ...  
8     use_data(p);  
9     ...  
10    free(p);  
11    ...  
12    use_data(q);           // q and p are aliased  
13                           // use after free  
14    ...  
15    free(q);              // double free  
16 }
```

- Every allocation creates an obligation: the memory must be released **exactly once**.
- It must be released on **every execution path**, including errors and early returns.
- Common failures:
 - **memory leak**: memory never released
 - **dangling pointer**: memory used after release
 - **double free**: memory released twice
- Aliasing makes ownership harder: Which pointer is responsible for freeing the object?
- Modern C++ reduces these risks through RAII and smart pointers; Python manages object lifetime automatically.

Heap Allocation and Garbage Collection

- In Python, objects are allocated dynamically on the heap.
- Names, list slots, dictionary entries, object fields, etc. hold references to objects in the heap.
- When an object becomes unreachable, Python can reclaim it automatically.
- CPython mainly uses reference counting, plus a cyclic garbage collector.

```
1 def f():  
2     x = [1, 2, 3]  
3     y = [x, x]  
4     return len(y)  
5  
6 f()  
7 # After f returns, x and y are no longer reachable. The garbage collector will eventually reclaim the memory.
```

- Contrast with C/C++: forgetting `free` / `delete` can leak memory.

References: [gc: Garbage Collector interface](#), [Data model: Objects, values and types](#).

Dictionaries (aka maps, hash-tables)



- Stores objects identified by keys.

```
1 >>> phonebook = {"bob": 7387, "alice": 3719, "jack": 7052}
```

- Dictionary comprehensions work like list comprehensions.

```
1 >>> phonebook = {name: number for name, number in zip(["bob", "alice", "jack"], [7387, 3719, 7052])}
```

- Access: Use a syntax similar to lists/arrays:

```
1 phonebook["bob"]
```

- The key object should be hashable.

References: [Dictionaries](#).

Dictionary operations

Common dictionary operations:

- `d.clear()` – Clears the dictionary `d`.
- `d.get(<key>[, <default>])` – Returns the value associated with `key`. If the key is not present in the dictionary, it returns `default`.
- `d.items()` – Returns a view of key: value pairs in a dictionary. Views are iterable.
- `d.keys()` – Returns a view of keys in a dictionary.
- `d.values()` – Similar.
- `d.pop(<key>[, <default>])` – Removes a key from a dictionary, if it is present, and returns its value.
- `d.update(<obj>)` – Merges a dictionary with another dictionary/list.

References: [Dictionaries](#).

Other Data Structures

- **Tuples:** Immutable containers.

```
1 >>> point_3d = (4.6, 5.7, -2.1)
```

- **array.array:** A C-like array with all elements having the same type.

```
1 >>> import array
2 >>> arr = array.array("f", (1.0, 1.5, 2.0, 2.5))
```

- These are mutable, dynamic, and homogeneous.
- They are more time and space efficient. Why?

- **Sets:** Unordered collections.

```
1 >>> vowels = {"a", "e", "i", "o", "u"}
```

- There is also a variant called a multiset.

References: [Python data structures](#). [Complexity of operations on Python data structures](#).

Functions

Functions are **first-class values**: they can be assigned, passed, returned, and stored just like other values.

A function definition

```
1 def f(x, y):  
2     return x**2 + y
```

Assign a function to a variable

```
1 >>> g = f  
2 >>> g(3, 2)
```

Create an unnamed function

```
1 >>> (lambda x, y: x**2 + y)(3, 2)
```

A Python lambda is restricted to one expression.

Pass a function as an argument

```
1 def square(x):  
2     return x**2  
3 >>> list(map(square, [1, 2, 3]))
```

Return a function as a result

```
1 def make_power(n):  
2     def power(x):  
3         return x**n  
4     return power  
5 >>> square = make_power(2)  
6 >>> square(5)
```

Hold functions in a container

```
1 def square(x):  
2     return x**2  
3 def cube(x):  
4     return x**3  
5 >>> funcs = [square, cube]  
6 >>> [fn(3) for fn in funcs]
```

Classes and Objects

class variable

```
1 class Account():  
2     init_message = "Welcome customer"
```

```
3 def __init__(self, account_holder, balance):  
4     self.account_holder = account_holder  
5     self.balance = balance
```

constructor

```
7 def show(self):  
8     print(self.account_holder)  
9     print(self.balance)
```

instance variable

```
11 def withdraw(self, amount):  
12     if amount <= self.balance:  
13         self.balance -= amount  
14         return self.balance  
15     else:  
16         return "Insufficient funds"
```

```
18 def deposit(self, amount):  
19     self.balance += amount  
20     return self.balance
```

```
1 >>> my_account = Account("Amitabha Sanyal", 1000)  
2 >>> my_account.show()  
3 Amitabha Sanyal  
4 1000  
5 >>> print(my_account.init_message)  
6 Welcome customer
```

Classes and Objects: Inheritance

```
1 class Protected_Account(Account):           # Derived from Account
2     """Modelling a password protected bank account"""
3     def __init__(self, passwd, account_holder, balance): # Constructor of derived class
4         super().__init__(account_holder, balance)      # Call base constructor
5         self.__password = passwd                       # Private member
6     def withdraw(self, amount, passwd):               # Modified withdraw method
7         if passwd == self.__password:
8             return super().withdraw(amount)           # Call base withdraw
9         else:
10            return "Transaction failed, wrong password"
11    def deposit(self, amount):                         # Note: show() not defined
12        return super().deposit(amount)
```

```
1 >>> pacc = Protected_Account("lksdj", "Amitabha Sanyal", 1000)
2 >>> pacc.show()
3 Amitabha Sanyal
4 1000
5 >>> pacc.withdraw(30, "lksd")
6 'Transaction failed, wrong password'
7 >>> print(pacc.__password)                       # This will raise an AttributeError,
```

Python Classes Are First-Class Objects

- A Python class is itself a runtime object.
- Therefore, a class can be passed to a function, stored in a container, returned as a result, and called to create instances.

```
1 class Account:
2     def __init__(self, holder, balance):
3         self.holder = holder
4         self.balance = balance
5
6 class SavingsAccount(Account):
7     interest_rate = 0.04
8
9 class CurrentAccount(Account):
10    overdraft_limit = 10000
```

```
1 def open_account(account_class,
2                   holder, balance):
3     return account_class(holder,
4                           balance)
5
6 a = open_account(SavingsAccount,
7                 "Asha", 5000)
8
9 b = open_account(CurrentAccount,
10                 "Ravi", 8000)
```

- Here SavingsAccount and CurrentAccount are passed as values.
- In C++, a class name denotes a **type**, not an ordinary runtime object; the closest analogue is usually a template parameter or a factory function.

Example 1: IPL Matches Scores Problem

The following input represents the scores of players in IPL matches.

```
1 5
2 CSKvsRCB:AnujRawat-48,DineshKarthik-38
3 RCBvsPBKS:ViratKohli-77,ShikharDhawan-45
4 RCBvsKKR:ViratKohli-83,VenkateshIyer-50
5 RRVsRCB:ViratKohli-113,JosButtler-100
6 RCBvsSRH:TravisHead-102,DineshKarthik-83
```

- The first line gives the number of match records.
- Each subsequent line contains match details in the format: matchName:player-score,player-score,...

Write a Python program that reads the input, constructs a dictionary mapping each match to its player scores, and produces a sorted list of players with their aggregate runs.

Expected Output:

```
1 {'CSKvsRCB': {'AnujRawat': 48, 'DineshKarthik': 38}, 'RCBvsPBKS': {'ViratKohli': 77, 'ShikharDhawan': 45},
  'RCBvsKKR': {'ViratKohli': 83, 'VenkateshIyer': 50}, 'RRVsRCB': {'ViratKohli': 113, 'JosButtler': 100}, 'RCBvsSRH':
  {'TravisHead': 102, 'DineshKarthik': 83}}
2 [( 'ViratKohli' , 273), ( 'DineshKarthik' , 121), ( 'TravisHead' , 102), ( 'JosButtler' , 100), ( 'VenkateshIyer' , 50),
  ( 'AnujRawat' , 48), ( 'ShikharDhawan' , 45)]
```

Example 1: Solution

```
1  num_matches = int(input())
2  match_dict = {}                                # The outer dictionary
3  aggregate = {}                                # The aggregate list
4  for _ in range(num_matches):                  # Each iteration processes a match
5      line = input().strip()                    # read a line from input, stripping leading and trailing whitespaces
6      match_name, scores_str = line.split(':') # Split the line into match name and scores
7      scores_list = scores_str.split(',')       # Split the scores into a list
8      match_scores = {}                        # The inner dictionary for a match
9      for score in scores_list:
10         # Each score is in the form "player-run"
11         player, run_str = score.split('-')
12         run = int(run_str)
13         match_scores[player] = run
14         aggregate[player] = aggregate.get(player, 0) + run
15     match_dict[match_name] = match_scores
16 # "sorted" returns a list of aggregated scores, sorted in descending order by runs
17 sorted_aggregate = sorted(aggregate.items(), key=lambda item: item[1], reverse=True)
18 # Display the outputs
19 print(match_dict)
20 print(sorted_aggregate)
```

Example 2

You are the head of an intelligence unit and you suspect one of your staff to be a spy. The only clue that you have is the suspect's diary, called `MyDiary.txt`, which contains, amongst a rambling text, some email addresses and phone numbers — including your own. Your hypothesis is that the suspect is a spy if he has more conversations with some contact other than yourself.

Write a Python program to:

- First, print your own occurrence frequency on a line in the following format:

```
1 my frequency: <frequency>
```

- Report all the frequent contacts in the format:

```
1 Spy alert <spy's contact> <frequency>
2 Spy alert ...
```

- Otherwise, conclude with:

```
1 All's well, no spy!!!
```

Example 2

Your program will be invoked like:

```
1 python3 spy.py <path-to-diary> <mycontact>
2 # mycontact could be an email or phone no.
```

Sample output:

```
1 my frequency: 7
2 Spy alert! cockroach@badguys.com 10
```

Example 2: Regular Expressions

- Here is a semi-formal description of an email id:

<email id> = <local part> @ <domain>

<local part> = one or more <alphanumeric>s separated by <dot_or_us>

<alphanumeric> = one or more letters [a-zA-Z] or digits [0-9]

<dot_or_us> = the character . or the character _

<domain> = one or more <alphanumeric>s separated by <dot>. The last group of <alphanumeric>s should consist of <alphabetic>s and should have at least two characters

<alphabetic> = one or more letters.

<phone_no > = Ten consecutive digits not starting with 0.

- **Sample email id:** fxps_ho.4@anhthu.org
- **Note:** Email and Number will be at a word boundary but may be surrounded or adjacent to punctuations.
- You have to use regular expressions to find the email IDs and phone numbers.

Example 2: Solution

- First import some libraries:

```
1 import sys
2 import re
3 import string
4 from collections import Counter
```

- Give names to regular expressions for email addresses and phone numbers. Note that `re_email` contains three grouped subexpressions: one capturing group and two non-capturing groups. `re_number` has one.

```
1 local_part = "(?:[A-Za-z0-9._%+-]+)"
2 domain     = r"(?:[A-Za-z0-9.-]+\.[A-Za-z]{2,})"
3 re_email   = r"\b(" + local_part + "@" + domain + r")\b"
4
5 re_number  = r'(\b [1-9][0-9]{9}\b )'          # \b...\b for matching at word boundaries only
```

Example 2.

- A `findall` based on `re_email` will return a list of triples, whereas a `findall` based on numbers will result in just a list of numbers.

```
1 >>> re.findall(re_email, 'as@cse.iitb.ac.in, amit23358@gmail.com')
2 ['as@cse.iitb.ac.in', 'amit23358@gmail.com']
3 >>> re.findall(re_number, '1234567890, 9999999999')
4 ['1234567890', '9999999999']
```

- We use list comprehension to extract the first element (the match of the outer grouping):

```
1 emails = []; numbers = []
2 fp = open(sys.argv[1], "r")
3 for line in fp.readlines():           # For each line in MyDiary.txt do
4     emails += re.findall(re_email, line) # Accumulate all emails on the line
5     numbers += re.findall(re_number, line) # Same for phone numbers.
```

Example 2.

- Here is an outline of the rest of the code:

```
1  count_emails = Counter(emails)      # Creates a dict {email: count}
2  count_numbers = Counter(numbers)
3  isNumber = sys.argv[2].isdigit()    # Is my contact phone or email?
4  spy_dict = {}                      # Initialize a dictionary of spies
5  isSpy = False                       # To track whether a spy has been found
6
7  if isNumber:
8      print('my frequency:', count_numbers[sys.argv[2]])
9      for no in list(set(numbers)):    # Makes the numbers unique
10         if count_numbers[no] > count_numbers[sys.argv[2]]:
11             isSpy = True
12             spy_dict[no] = count_numbers[no]
13 else:
14     ...                               # Other actions
15 # Do the same for emails
```

- Now it is a simple matter to write the rest of the program.

Exceptions

- **Exception Handling:**

- Errors occur for various reasons: invalid arithmetic, malformed user input, missing files, invalid indices or keys, and failed or timed-out network connections.
- Exception handling makes programs more robust by detecting errors and responding meaningfully, rather than allowing the program to fail unexpectedly.

Syntax:

```
1 try:
2     # Code that might raise an exception
3 except SomeException as e:
4     # Code to execute if that exception occurs
```

Example:

```
1 try:
2     int("abc")
3 except ValueError as e:
4     print(e)
5
6 invalid literal for int() with base 10: 'abc'
```

Repeated retrials after recovery from:

```
1 import time
2
3 for attempt in range(3):
4     try:
5         result = contact_server()
6         break
7     except TimeoutError:
8         print("Server did not respond; retrying...")
9         time.sleep(1)
10 else:
11     print("Server unavailable after three attempts.")
```

Exception: Examples

- except, else and finally clauses:

```
1  try:
2      # Code that might raise multiple types of errors
3  except FileNotFoundError:
4      print("File not found!")
5  except ValueError:
6      print("Invalid value!")
```

```
1  try:
2      result = 10 / 2
3  except ZeroDivisionError:
4      print("Division by zero!")
5  else:
6      print("Result is", result)
7  finally:
8      # This block is always executed
9      # Insert cleanup code here
10     pass
```

```
1  def bar():
2      print("bar: starting")
3      1 / 0
4  def foo():
5      try:
6          print("foo: entering")
7          bar()
8          print("foo: after bar")      # never
                                         executed
9      finally:
10         print("foo: cleanup in finally")
11  try:
12     foo()
13  except ZeroDivisionError:
14     print("outer level: exception caught")
15  finally:
16     print("Cleanup actions executed")
```

Exceptions and Flow of Control

- When an error occurs in the `try` block, Python immediately stops executing the remaining code in that block.
- The interpreter then searches for a matching `except` clause to handle the raised exception.
- This process involves **stack unwinding**: Python pops stack frames (i.e., exits from functions) until it finds an appropriate exception handler.
- During stack unwinding, any `finally` blocks and context manager clean-up code are executed to release resources.
- If no matching `except` block is found, the exception propagates to the top level, and the program terminates with a traceback.

Predefined Exceptions in Python

- **Exception:** Base class for all built-in exceptions.
- **IndexError:** Raised when a sequence index is out of range.
- **KeyError:** Raised when a dictionary key is not found.
- **TypeError:** Raised when an operation is applied to an inappropriate type.
- **ValueError:** Raised when an argument has the right type but an inappropriate value.
- **ZeroDivisionError:** Raised when division or modulo by zero occurs.
- **FileNotFoundError:** Raised when a file or directory is requested but does not exist.
- **ImportError:** Raised when an import statement fails to load a module.

For a complete list, visit: <https://docs.python.org/3/library/exceptions.html>

Modules and Imports

- A **module** is a .py file containing definitions and statements.
- Importing a module gives access to the names defined in it:
 - function names, class names, variable names, imported module names, etc.
 - these names live in the module's own namespace.

```
1 # stats.py
2 version = "1.0"
3 def mean(xs):
4     return sum(xs) / len(xs)
```

```
1 # use_stats.py
2 import stats
3
4 marks = [8, 9, 10]
5 print(stats.mean(marks))
6 print(stats.version)
```

```
1 # use_stats.py
2 from numpy import *    # Also defines a mean
3 from stats import *
4 marks = [8, 9, 10]
5 print(mean(marks))
6 print(version)
```

- Prefer `import stats`: Keeps the origins of the names visible.
- Avoid `from modulename import *`: it imports an unknown set of names.

Inspecting Namespaces

Example module

```
1  # inspect_demo.py
2  import math
3  x = 10
4  def f(a):
5      local = a + 1
6      return locals()
7  def show():
8      return globals()
9  class C:
10     y = 20
11     def method(self):
12         print(locals())
13         return self.z + C.y
14  obj = C()
15  obj.z = 30
```

Try these inspections

```
1  >>> inspect_demo.show()["x"]
2  10
3  >>> "math" in inspect_demo.show()
4  True
5  >>> inspect_demo.f(5)
6  {'a': 5, 'local': 6}
7  >>> inspect_demo.C.__dict__["y"]
8  20
9  >>> inspect_demo.C.__dict__
10 ... 'y': 20, 'method': <function C.method at 0x7278488f6710>...
11 >>> vars(inspect_demo.obj)
12 {'z': 30}
13 >>> "method" in dir(inspect_demo.obj)
14 True
```

- `globals()` and `locals()` inspect namespaces.
- `__dict__` and `vars()` expose attribute mappings.
- `dir()` lists available names; it does not return their values.

Packages, Standard Library, and PyPI

- A **package** organizes related modules using dotted names.

```
1 project/  
2   analysis/  
3     __init__.py  
4     io.py  
5     stats.py  
6     main.py
```

```
1 from analysis import stats  
2 from analysis.io import read_marks
```

- The **standard library** is shipped with Python: `sys`, `os`, `pathlib`, `re`, `collections`, `json`, `argparse`, ... Third-party packages are commonly installed from **PyPI**.
- A regular package is a directory with `__init__.py`; importing the package executes this file and initializes the package namespace.

References: [Packages](#), [The Python Standard Library](#), [Packaging Python Projects](#), [PyPI](#).

Reading and Writing Files

- Use `with open(...)` so files are closed automatically.
- Text mode returns strings; binary mode returns bytes.

```
1  # Text file
2  with open("marks.txt", "r", encoding="utf-8") as f:
3      for line in f:
4          roll, marks = line.split()
5          print(roll, int(marks))
6
7  # Binary file
8  with open("image.png", "rb") as f:
9      header = f.read(8)
```

- Common modes: "r", "w", "a", "rb", "wb".

References: [Reading and Writing Files](#), [open](#).

Paths with pathlib

- Prefer `pathlib.Path` over manual string manipulation.
- Path operations become object operations.

```
1  from pathlib import Path
2
3  data_dir = Path("data")
4  input_file = data_dir / "marks.txt"
5  output_file = data_dir / "summary.txt"
6
7  print(input_file.name)      # marks.txt
8  print(input_file.suffix)    # .txt
9  print(input_file.exists())  # False
10
11 with input_file.open("r", encoding="utf-8") as f:
12     lines = f.readlines()
13
14 output_file.write_text("Number of lines: " + str(len(lines)) + "\n",
15                        encoding="utf-8")
```

Command-line Arguments with `sys.argv`

- Small scripts often receive filenames from the command line.
- `sys.argv` is the list of command-line arguments.
- `sys.argv[0]` is the script name; later entries are user arguments.

```
1 # count_lines.py
2 from pathlib import Path
3 import sys
4
5 if len(sys.argv) != 2:
6     print("Usage: python3 count_lines.py FILE")
7     sys.exit(1)
8 path = Path(sys.argv[1])
9
10 with path.open("r", encoding="utf-8") as f:
11     nlines = sum(1 for line in f)
12
13 print(path, "has", nlines, "lines")
```

References: [sys.argv](#), [argparse: Parser for command-line options](#).

Debugging

- Use `assert` for conditions that must be true.
- Use `breakpoint()` to enter the debugger at a chosen point, or do a postmortem debugging.

```
1 import pdb
2 def divide(total, count):
3     assert(count > 0), "Count must be positive"
4     return total / count
5 def average(scores):
6     return divide(sum(scores), len(scores))
7 def parse_record(line):
8     name, raw = line.split(":")
9     scores = [int(x) for x in raw.split(",") if x]
10    return name, average(scores)
11 def main():
12     lines = ["Amit:80,90", "Vikram:"]
13     for line in lines:
14         name, avg = parse_record(line)
15         print(name, avg)
16 main()
```

```
1 python3 -m pdb program.py
2
3 args          function arguments and their values
4 p expr        evaluate and print an expression
5 list          show nearby source lines
6 where         show the call stack
7 up/down       select another stack frame
8 next          next line, step over function calls
9 step          next line, enter function calls
10 break 20      set a breakpoint at line 20
11 break 20, x==0 conditional breakpoint
12 continue      resume execution
13 quit          terminate debugging
14 help          list commands
15 locals()      local variables and args
16 display x
```

- Good debugging recipe: reproduce, minimize, inspect, fix, retest.

Virtual Environments and pip

- A virtual environment keeps project packages separate from system Python.
- Avoid installing project dependencies globally.

```
1 $ python3 -m venv .venv
2 $ source .venv/bin/activate
3 $ (.venv) which python
4 /home/as/.../.venv/bin/python
5 $ (.venv) python -m pip install requests pandas
6 $ (.venv) python -m pip freeze > requirements.txt
7 $ (.venv) deactivate
8 $
```

- Use `python -m pip` so pip belongs to the selected Python.

References: [venv: Creation of virtual environments](#), [Installing Python Modules](#).